

Labolatorium 2 - Uwagi, problemy, todo list

Napotkane (niektóre/wybrane) problemy

- Przy włączaniu schematica `DAC_CORE_5bit_test.sch` schemat potrafi się zablokować i nie dać możliwości do edycji pliku. Dzieje się tak zazwyczaj gdy użytkownik chce edytować parametry i zamknie okno dialogowe edytowania parametrów. Na początku nie było tego problemu - musiał wystąpić po którymś z `sudo apt update && sudo apt upgrade -y` i restartach pc.
Błąd w konsoli:

```
xschem [/foss/designs/lab2/xschem] tclevel(): evaluation of script: edit_prop
{Input property:} failed
: window ".dialog" was deleted before its visibility changed
```

Rozwiązanie: Zauważyłem, że dzieje się tak gdy użytkownik przed edycją nie stworzy netlisty schematu przed edycją - czyli po włączeniu schematu należy zbudować netlistę. Nie znalazłem jeszcze konkretnego powodu dlaczego to ma wpływ, więc jest to chwilowe rozwiązanie.

TBD

TODO LIST

1. Znaleźć sposób na uwzględnienie rozrzutów tylko pojedynczego tranzystora i uzupełnić punkty 1.5 - 1.7 (znalazłem sposób - trzeba uzupełnić instrukcję) - **zrobione**
 - To co Defaultowo jest przez `SKY130->Add models symbol` może być zmienione poprzez modyfikację `/foss/pdks/sky130A/libs.tech/xschem/xschemrc`, ale to nie będzie permanentna zmiana, a wyłącznie na obecną sesję dockera, do stałej zmiany można dodać plik `/foss/designs/xschemrc` **POD WARUNKIEM że będziemy startować każdy schematic z folderu /foss/designs co przy dużej ilości podfolderów i plików, czy kilkusobowej pracy nad jednym projektem, może być niewygodne (trzeba być zapoznanym z drzewem folderów)**
 - dla bardzo małych prądów, mogą powstawać niedokładności między dużą ilością powtórzeń. Czemu? Może to przez niedokładności w modelowaniu? to raczej nie jest problem tylko z .subckt - dla pfetów prosto z sky130 też tak to wygląda - sprawdzić jeszcze trzeba i się zastanowić
2. Przeprowadź analizę MC z punktu 1.8 i zapisz wyniki (kod do symulacji gotowy - ustawić M_REF jako mosfet idealny) - **zrobione**
3. Zaktualizuj polecenie 1.10 uwzględniając odpowiednie parametry i sposoby na rozrzucanie pojedynczego tranzystora - **zrobione**
4. Wykonaj symulacje zgodnie z punktem 1.11 - **zrobione**
5. **5BIT DAC SCHEMATIC** - **zrobione**
6. Stwórz symbol dla 5bit dac - **zrobione**
7. Stwórz schemat symulacyjny - **zrobione**
8. Przeprowadź symulacje działania układu - VTH jest na poziomie 1V dla W/L= 2/5 przez co nie jesteśmy w stanie uzyskać prądu bliskiego 10uA, nie bawić się w lvt-pfets, dla W/L= 2/5 vgs ~ 1.9V dla zasilania 1.8V X_X - modyfikacja W/L, **obecnie 3.5/5**, bez problemu można by było zostać przy W = 2 i zmniejszać L do momentu uzyskania odpowiedniego prądu - **zrobione**

9. Zastanów się jak zrobić symulacje cornerowe dla każdego przypadku (chodzi o każdy corner w jednej symulacji - czy jest to wogóle możliwe, jeśli nie - po prostu zmieniaj corner co symulację) - zostałem przy drugiej opcji, pierwsza do sprawdzenia, **zrobione**

10. Layout DAC-a - **zrobione**

- Klayout jest dość intuicyjny, ale importowanie z netlisty wymaga aktualizacji "Packages", co trzeba robić za każdym odpaleniem dockera - nie znalazłem jeszcze jednoznacznego rozwiązania, dzięki któremu mógłbym mieć najnowsze makra dla każdej instancji dockera (.designinit w folderze design - do testu; widzę że na gitcie iic osic tools jest nadal rozwijane, w zeszłym tygodniu był commit o updatecie niektórych narzędzi na branch next_release);
- tworzenie własnych makr (wymaga wiedzy o samym klayout i pdk'ach; kodowanie w pythonie) i dostęp do utworzonych przez użytkowników na plus - obecnie nie jest ich wiele ale, niektóre są przydatne, pomagają w nauce oprogramowania (przykłady: import from netlist, graphical layout keybindings);
- najlepiej od razu odpalać klayout w trybie do edycji `klayout -e`, podobny problem jak z makrami da się zmienić żeby default launch był w trybie edycji, ale jak zrobić żeby zmiana została zapisana (znowu .designinit, albo klayoutrc, modyfikacja samego iic tools albo skryptu startowego?,)

11. Ekstrakcja parasitic - **zrobione**

1. klayout-pex tool:

```
kpex --gds DAC_CORE_5bit.gds \  
--cell DAC_CORE_5bit \  
--pdk sky130A \  
--magic \  
--mode RC \  
--schematic /foss/designs/lab2/xschem/DAC_CORE_5bit.spice \  
--out_spice DAC_CORE_5bit_pex.spice
```

2. magic extract:

```
magic -dnull -nocmd  
gds read DAC_CORE_5bit.gds  
load DAC_CORE_5bit  
extract all  
ext2sim labels on  
ext2sim  
extresist tolerance 10  
extresist  
ext2spice lvs  
ext2spice ground VDD  
ext2spice defaultsub VDD  
ext2spice cthresh 0.1  
ext2spice extresist on  
ext2spice -o DAC_CORE_5bit_mpex.spice  
exit
```

3. !!! SPRAWDŹ CZY ZGADZA SIĘ KOLEJNOŚĆ PINÓW MIĘDZY SYMBOLEM ZE SCHEMATU A .SUBCKT W PLIKU _PEX !!!

12. Co do samej instrukcji - dodać więcej zdjęć (łatwiej się pracuje z referencyjnymi zdjęciami) lub wstawiać pomocnicze fragmenty kodu (większość ustawiania to po prostu ngspice code)
13. W punkcie 3.10. była uwaga - "zwróć uwagę na etykiety pomiędzy kluczami a źródłami prądowymi. Bez tych etykiet możesz mieć problem z analizą LVS." - nie zauważyłem żeby to było problemem dla sky130 LVS:
 - jeżeli schemat będzie miał podpisy `M*_C`, a na layoutcie nie podpiszemy / nie użyjemy warstwy label dla podpisania tych netów to na podstawie instancji LVS automatycznie je wykryje i przypisze (Rys 3.xx.).
 - Jeżeli nie podpiszemy netów w schemacie to spice i tak musi mieć nazwy na nety i zostanie po prostu `net*` co znowu - zostanie automatycznie wykryte przez LVS.
14. Żeby podłączyć B do VDD należy rozlać *Nwell* pod całą matrycą ORAZ guard ringiem (tak, żeby położony *Nwell* pokrywał również *nsdm* z generowanego guard ringa)